```
//Validator<User> nameValidator = Validator.field(
//    "name",
//    User::getName,
//
Validator.<String>notNull().and(Validator.minLength(3))
//);
public static <T, R> Validator<T> field(
        String name,
        Function<T, R> getter,
        Validator<R> validator) {

    if (name == null || getter == null || validator ==
null) {
        throw new IllegalArgumentException();
    }

    return value -> {
        R fieldValue = getter.apply(value);

        ValidationResult result =
validator.validate(fieldValue);

        if (result.isValid()) {
            return ValidationResult.ok();
        }

        List<String> errors = new ArrayList<>();

        for (String error : result.getErrors()) {
            errors.add(name + ": " + error);
        }

        return ValidationResult.of(errors);
    };
}



//Validator<User> userValidator = Validator.allOf(
//    List.of(nameValidator, ageValidator,
emailValidator)
//);
```

```
public static <T> Validator<T> allOf(
        List<Validator<T>> validators) {

    if (validators == null) {
        throw new IllegalArgumentException();
    }

    for (Validator<T> validator : validators) {
        if (validator == null) {
            throw new IllegalArgumentException();
        }
    }

    return value -> {
        List<String> errors = new ArrayList<>();

        for (Validator<T> validator : validators) {
            ValidationResult result =
validator.validate(value);

            errors.addAll(result.getErrors());
        }

        return ValidationResult.of(errors);
    };
}


public static ValidationResult validate(User user) {
    if (user == null) {
        throw new IllegalArgumentException();
    }

    Validator<User> nameValidator = Validator.field(
            "name",
            User::getName,
            Validator.<String>notNull()
                    .and(Validator.minLength(3))
    );

    Validator<User> ageValidator = Validator.field(
            "age",
            User::getAge,
            Validator.<Integer>notNull()
```

```
                    .and(Validator.range(18, 120))
    );

    Validator<User> emailValidator = Validator.field(
            "email",
            User::getEmail,
            Validator.<String>notNull()
                    .and(Validator.minLength(5))
                    .and(email -> email.contains("@")
                            ? ValidationResult.ok()
                            :
ValidationResult.error("must contain @"))
    );

    return Validator.allOf(
            List.of(nameValidator, ageValidator,
emailValidator)
    ).validate(user);
}
```